

C2SafeRust-LLM

Module 2: Preprocessing & AST Construction -- Line-by-Line Explanation

For Beginners | Mini Project S3

Module 2: Preprocessing & AST Construction

Line-by-Line Code Explanation for Beginners

Files: `src/module2_ast/mod.rs`, `preprocessor.rs`, `ast_parser.rs`

What does Module 2 do? Module 2 takes the raw C text validated by Module 1 and converts it into a structured tree of data that the rest of the pipeline can work with programmatically. It has TWO jobs: 1. **PREPROCESSING**: Expand macros (`#define`), resolve `#ifdef` blocks, strip `#include` headers 2. **AST CONSTRUCTION**: Parse the expanded C code into a tree of structs, functions, unions The output is a **TranslationUnitAST** — a Rust data structure representing the entire C program organized as functions, structs, and unions.

```
ValidatedCSource (from Module 1)
  |
  v  preprocessor.rs
ExpandedCSource (macros resolved, headers stripped)
  |
  v  ast_parser.rs
TranslationUnitAST (structured tree of functions/structs/unions)
```

File 1: `src/module2_ast/mod.rs`

(The Orchestrator — Glues Preprocessor and AST Together)

Lines 1-5: Sub-module declarations and re-exports

```
pub mod preprocessor;
pub mod ast_parser;

pub use preprocessor::{preprocess_c_code, ExpandedCSource, PreprocessorError};
pub use ast_parser::{
    construct_ast, TranslationUnitAST, FunctionAST, ParameterAST,
    CallSiteAST, UnionAST, StructAST, ASTError
};
```

Line	Code	Explanation
<code>`pub mod preprocessor;`</code>	Sub-module declaration	Declares that <code>`preprocessor.rs`</code> is part of this module. Other modules can use <code>`module2_ast::preprocessor::...`</code> .
<code>`pub mod ast_parser;`</code>	Sub-module declaration	Same for <code>`ast_parser.rs`</code> .
<code>`pub use preprocessor::{...};`</code>	Re-export	Brings types FROM <code>`preprocessor.rs`</code> INTO the top level of <code>`module2_ast`</code> . So other modules can write <code>`module2_ast::ExpandedCSource`</code> instead of <code>`module2_ast::preprocessor::ExpandedCSource`</code> . This is a convenience shortcut.
<code>`pub use ast_parser::{...};`</code>	Re-export	Same convenience re-export for all AST types.

Line 7: Cross-module import

```
use crate::module1_validation::ValidatedCSource;
```

Code	Explanation
<code>`crate::`</code>	Refers to the ROOT of the current project (the crate).
<code>`module1_validation::ValidatedCSource`</code>	Import the output type from Module 1. Module 2 takes Module 1's output as its input.

Lines 9-13: The Combined Output Struct

```
#[derive(Debug, Clone)]
pub struct ProcessedModule2Output {
    pub expanded_source: ExpandedCSource,
    pub ast: TranslationUnitAST,
}
```

This struct bundles together BOTH things Module 2 produces: the preprocessed source AND the parsed AST. It is returned by `preprocess_and_parse()`.

Field	Type	What it stores
<code>`expanded_source`</code>	<code>`ExpandedCSource`</code>	The preprocessed C code (macros expanded, <code>#ifdefs</code> resolved).
<code>`ast`</code>	<code>`TranslationUnitAST`</code>	The full structured AST parsed from the expanded source.

Lines 15-27: The Main Entry Point `preprocess_and_parse()`

```
pub fn preprocess_and_parse(
    validated: &ValidatedCSource
) -> Result<ProcessedModule2Output, String> {
    let expanded = preprocess_c_code(validated)
        .map_err(|err| format!("Preprocessor Error: {}", err));

    let ast = construct_ast(&expanded)
        .map_err(|err| format!("AST Parse Error: {}", err));

    Ok(ProcessedModule2Output {
        expanded_source: expanded,
        ast,
    })
}
```

	Code	Explanation		
CSource;`	Borrows Module 1's output	Takes a reference (doesn't take ownership). Module 1's struct stays alive.		
	Return type	Success returns the combined output. Failure returns a plain `String` error message (simpler than a custom enum here).		
validated)`	Step 1	Calls the preprocessor. Returns `Result`.		
	err\	format!(...))`	Error conversion	Converts `Pr plain `String`
	Propagate error	If preprocessing fails, immediately return `Err(String)` upward.		
led;)`	Step 2	Calls the AST parser on the expanded source.		
output {	Return success	Bundle both outputs together and return.		

Pipeline Pattern: Notice how Module 2 follows a clean pipeline: Input -> Preprocess -> Parse -> Output Each step uses ? to propagate errors, making the "happy path" easy to read.

File 2: src/module2_ast/preprocessor.rs

(Expanding Macros and Resolving Directives)

What is preprocessing in C? Before C code is compiled, a "preprocessor" runs first. It handles: - #define F00 42 — replace every F00 with 42 in the code - #include <stdio.h> — paste in header file contents - #ifdef DEBUG ... #endif — include or exclude code blocks based on flags In our project, we need the EXPANDED version (after preprocessing) for accurate parsing.

Lines 1-3: Imports

```
use std::collections::HashMap;
use std::fmt;
use std::process::Command;
use crate::module1_validation::ValidatedCSource;
```

Import	Purpose
<code>`HashMap`</code>	Store macro definitions: <code>"BUFFER_SIZE" -> "1024"</code> .
<code>`fmt`</code>	Implement <code>`Display`</code> for <code>`PreprocessorError`</code> .
<code>`std::process::Command`</code>	Launch external programs (GCC) as child processes.
<code>`ValidatedCSource`</code>	The input from Module 1.

Lines 6-11: The Output Struct ExpandedCSource

```
#[derive(Debug, Clone)]
pub struct ExpandedCSource {
    pub raw_input: ValidatedCSource,
    pub expanded_content: String,
    pub used_external_preprocessor: bool,
}
```

Field	What it stores
<code>`raw_input`</code>	The original Module 1 output (kept for reference, e.g., the original file path).
<code>`expanded_content`</code>	The preprocessed C code as a String — macros replaced, <code>#ifdef</code> blocks resolved.
<code>`used_external_preprocessor`</code>	<code>`true`</code> if <code>`gcc -E`</code> was used; <code>`false`</code> if the internal fallback was used. Shown in the pipeline output.

Lines 13-24: The Error Enum and Display

```
#[derive(Debug, Clone)]
pub enum PreprocessorError {
    ExpansionFailed(String),
}

impl fmt::Display for PreprocessorError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            PreprocessorError::ExpansionFailed(msg) =>
                write!(f, "Preprocessor Expansion Error: {}", msg),
        }
    }
}
```

Only one error variant here because preprocessing can only fail one way: the expansion process failed (GCC crashed, or the internal preprocessor couldn't handle the input).

Lines 28-46: The Main Entry Point `preprocess_c_code()`

```
pub fn preprocess_c_code(
    input: &ValidatedCSource
) -> Result<ExpandedCSource, PreprocessorError> {
    // Attempt external gcc -E preprocessing first
    if let Ok(expanded) = try_gcc_preprocessor(input) {
        return Ok(ExpandedCSource {
            raw_input: input.clone(),
            expanded_content: expanded,
            used_external_preprocessor: true,
        });
    }

    // Fallback to internal custom preprocessor
    let expanded = fallback_internal_preprocessor(&input.raw_content)?;
    Ok(ExpandedCSource {
        raw_input: input.clone(),
        expanded_content: expanded,
        used_external_preprocessor: false,
    })
}
```

Section	Explanation
<code>`if let Ok(expanded) = try_gcc_preprocessor(input)`</code>	Try GCC first. <code>`if let Ok(x) = y`</code> extracts the success value if <code>`y`</code> returned <code>`Ok`</code> . If GCC fails (returns <code>`Err`</code>), skip to fallback.
<code>`return Ok(ExpandedCSource { ..., used_external_preprocessor: true })`</code>	GCC succeeded. Bundle result and return immediately.
<code>`fallback_internal_preprocessor(&input.raw_content)?`</code>	GCC failed. Run the built-in preprocessor instead. The <code>`?`</code> propagates any error.
<code>`used_external_preprocessor: false`</code>	Mark that GCC was NOT used.

Why prefer GCC? GCC's preprocessor handles ALL C preprocessing correctly including complex macros, system headers, and platform-specific code. Our internal fallback handles only simple cases. We try GCC first (best quality) and fall back only if GCC isn't installed.

Lines 48-66: `try_gcc_preprocessor()` — Calling GCC

```
fn try_gcc_preprocessor(
    input: &ValidatedCSource
) -> Result<String, PreprocessorError> {
    let output = Command::new("gcc")
        .arg("-E")
        .arg("-P")
        .arg(&input.file_path)
        .output()
        .map_err(|e| PreprocessorError::ExpansionFailed(e.to_string()))?;

    if output.status.success() {
        let content = String::from_utf8_lossy(&output.stdout).to_string();
        if !content.trim().is_empty() {
            return Ok(content);
        }
    }

    Err(PreprocessorError::ExpansionFailed(
        "GCC preprocessor failed or produced empty output.".to_string(),
    ))
}
```

	Code	Explanation		
	Create a system command	Starts building a command to run the `gcc` program. Like typing `gcc` in a terminal.		
	Flag: preprocess only	`gcc -E` means "run ONLY the preprocessor, don't compile". Outputs expanded C code.		
	Flag: suppress line markers	Without `-P`, GCC inserts `# 1 "file.c"` line markers into the output. `-P` removes these, giving us clean code.		
h)`	The input file	Passes the C file path as the argument to GCC.		
	Run the command	Executes GCC as a child process and waits for it to finish. Returns `Result`.		
	e\	...)`	Error conversion	If GCC can't convert the I
s())`	Check exit code	GCC returns exit code 0 on success. `.success()` returns `true` if exit code is 0.		
ssy(&outpu	Read output	GCC's preprocessed output comes through `stdout` as raw bytes. Convert to a String, replacing invalid UTF-8 with `?`.		
pty())`	Non-empty check	Even if GCC succeeded, make sure it actually produced output.		

Lines 68-133: fallback_internal_preprocessor() — Built-in Macro Expander

```
fn fallback_internal_preprocessor(
    c_code: &str
) -> Result<String, PreprocessorError> {
    let mut macros: HashMap<String, String> = HashMap::new();
    let mut processed_lines = Vec::new();
    let mut skip_stack = Vec::new();
    // ...
}
```

Variable	Type	Purpose
<code>`macros`</code>	<code>`HashMap`</code>	Dictionary of macro definitions. Key = macro name, Value = replacement text. E.g., <code>`"BUFFER_SIZE" -> "1024"`</code> .
<code>`processed_lines`</code>	<code>`Vec`</code>	Accumulates the output lines after processing.
<code>`skip_stack`</code>	<code>`Vec`</code>	A stack tracking <code>`#ifdef`/`#ifndef`</code> nesting. <code>`true`</code> = currently skipping (inactive branch).

Processing `#ifdef` / `#ifndef` / `#else` / `#endif`:

```
if trimmed.starts_with("#ifdef") {
    let macro_name = trimmed.trim_start_matches("#ifdef").trim();
    let is_defined = macros.contains_key(macro_name);
    skip_stack.push(!is_defined); // Push true if macro NOT defined (skip this branch)
    continue;
} else if trimmed.starts_with("#ifndef") {
    let macro_name = trimmed.trim_start_matches("#ifndef").trim();
    let is_defined = macros.contains_key(macro_name);
    skip_stack.push(is_defined); // Push true if macro IS defined (skip this branch)
    continue;
} else if trimmed.starts_with("#else") {
    if let Some(top) = skip_stack.pop() {
        skip_stack.push(!top); // Flip the skip state
    }
    continue;
} else if trimmed.starts_with("#endif") {
    skip_stack.pop(); // End of conditional block
    continue;
}
```

Logic	Example		
If <code>`FOO`</code> is in the macros dictionary, DON'T skip. If not defined, push <code>`true`</code> (skip).	<code>`#ifdef DEBUG`</code> when <code>`DEBUG`</code> is defined → include code		
OPPOSITE of <code>`#ifdef`</code> . If <code>`FOO`</code> IS defined, skip this branch.	<code>`#ifndef NDEBUG`</code> is common for debug guards		

	Logic	Example		
	Flip the skip state: if we were skipping, now include; if including, now skip.	Switch to the other branch		
	Pop the top of the stack — the conditional block is done.	End of the <code>`#ifdef`</code> block		
<code>/\</code>	<code>&skip;\</code>	<code>skip)`</code>	If ANY level of the nested stack says "skip", skip the current line.	Handles nes

Processing `#define`:

```

if trimmed.starts_with("#define") {
  let parts: Vec<&str> = trimmed.split_whitespace().collect();
  if parts.len() >= 2 {
    let name = parts[1].to_string();
    let val = if parts.len() >= 3 {
      parts[2..].join(" ")
    } else {
      "1".to_string()
    };
    macros.insert(name, val);
  }
  continue;
}

```

Line	Code	Explanation
<code>`trimmed.split_whitespace().collect()`</code>	Split by spaces	<code>`#define BUFFER_SIZE 1024`</code> splits into <code>`["#define", "BUFFER_SIZE", "1024"]`</code> .
<code>`parts[1]`</code>	Macro name	<code>`"BUFFER_SIZE"`</code>
<code>`parts[2..].join(" ")`</code>	Macro value	Everything from index 2 onward joined with spaces. Handles multi-word values like <code>`#define MSG "hello world"`</code> .
<code>`"1".to_string()`</code>	Flag macros	<code>`#define DEBUG`</code> with no value gets value <code>`"1"`</code> .
<code>`macros.insert(name, val)`</code>	Store it	Add to the dictionary for later substitution.

Processing `#include` and performing substitutions:

```

if trimmed.starts_with("#include") {
  continue; // Skip ALL #include lines
}

let mut expanded_line = line.to_string();
for (name, val) in &macros {
  expanded_line = expanded_line.replace(name, val);
}
processed_lines.push(expanded_line);

```

	Explanation	
<code>_with("#include") { continue; }</code>	Skip all <code>#include</code> directives. We can't paste header files in our simple preprocessor, so we just ignore them. GCC handles this properly.	
<code>replace(name, val)</code>	Macro substitution	For each known macro, replace its name with the value. E.g., <code>"int buf[BUFFER_SIZE];"</code> becomes <code>"int buf[1024];"</code> .
<code>push(expanded_line)</code>	Save the processed line	Add the (possibly substituted) line to the output.

File 3: `src/module2_ast/ast_parser.rs`

(Building the Abstract Syntax Tree)

What is an AST? An Abstract Syntax Tree (AST) is a tree-shaped data structure that represents the *STRUCTURE* of source code — not the raw text, but what it *MEANS*. Example: `void swap(int **a, int **b) { ... }` becomes: `FunctionAST { name: "swap", return_type: "void", parameters: [ ParameterAST { name: "a", c_type: "int", pointer_depth: 2, ... }, ParameterAST { name: "b", c_type: "int", pointer_depth: 2, ... } ], body_code: "...", call_sites: [...] }`

Lines 1-2: Imports

```
use std::fmt;
use crate::module2_ast::preprocessor::ExpandedCSource;
```

- `fmt` — for implementing `Display` on `ASTError`.
- `ExpandedCSource` — the AST parser's input (from the preprocessor).

The AST Node Types (Lines 4-63)

These structs are the "building blocks" of the AST. Each represents one concept in C code.

`ParameterAST` — One function parameter (Lines 4-10)

```
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct ParameterAST {
    pub name: String,
    pub c_type: String,
    pub pointer_depth: usize,
    pub is_union: bool,
}
```

Represents *ONE* parameter in a function signature. For `void swap(int **a, int **b)`, there are *TWO* `ParameterAST` instances.

Field	Example for <code>int **a</code>	Explanation
<code>`name`</code>	<code>"a"</code>	The parameter's variable name.
<code>`c_type`</code>	<code>"int"</code>	The BASE type (without the <code>**</code> stars).
<code>`pointer_depth`</code>	<code>2</code>	How many <code>**</code> stars: <code>int*</code> = 1, <code>int**</code> = 2, <code>int</code> = 0.
<code>`is_union`</code>	<code>false</code>	<code>true</code> if the type contains the word <code>union</code> (e.g., <code>union Data *u</code>).

Why track `pointer_depth` separately? `int` → Rust `i32` (no stars = plain value) `int*` → Rust `&i32` or `&mut i32` (one star = reference) `int**` → Rust `Box<i32>` or `&mut &mut i32` (two stars = complex ownership)
The depth number is crucial for the ownership inference in Module 5.

CallSiteAST — One function call (Lines 12-17)

```
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct CallSiteAST {
    pub callee: String,
    pub arguments: Vec<String>,
    pub line: usize,
}
```

Represents one occurrence of calling a function inside a function body. For `swap(&px, &py);` on line 15, this becomes: `CallSiteAST { callee: "swap", arguments: [&px, &py], line: 15 }`

Field	Example	Explanation
<code>`callee`</code>	<code>"swap"</code>	The name of the function being called.
<code>`arguments`</code>	<code>["&px;", "&py;"]</code>	The raw argument strings as they appear in the source.
<code>`line`</code>	<code>15</code>	Which line number the call is on — used to build the call graph edge in Module 4.

FunctionAST — One complete function (Lines 19-26)

```
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct FunctionAST {
    pub name: String,
    pub return_type: String,
    pub parameters: Vec<ParameterAST>,
    pub body_code: String,
    pub call_sites: Vec<CallSiteAST>,
}
```

The most important AST node — represents one complete C function. Every function in the C file gets its own **FunctionAST** instance.

Field	Example for <code>void swap(int **a)</code>	Explanation
<code>`name`</code>	<code>"swap"</code>	The function's name.
<code>`return_type`</code>	<code>"void"</code>	What the function returns.
<code>`parameters`</code>	<code>[ParameterAST{name:"a",...}]</code>	List of all parameters.
<code>`body_code`</code>	<code>"int *temp = *a;\n*a = *b;..."</code>	The entire function body as raw text.
<code>`call_sites`</code>	<code>[CallSiteAST{callee:"swap",...}]</code>	All function calls made inside this function's body.

UnionVariantAST and UnionAST — C union definitions (Lines 28-39)

```
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct UnionVariantAST {
    pub field_name: String,
    pub field_type: String,
}

#[derive(Debug, Clone, PartialEq, Eq)]
pub struct UnionAST {
    pub name: String,
    pub variants: Vec<UnionVariantAST>,
    pub tag_field: Option<String>,
}
```

For a C union like: `c union Data { int i; float f; };` We get: UnionAST { name: "Data", variants: [UnionVariantAST{field_name:"i", field_type:"int"}, UnionVariantAST{field_name:"f", field_type:"float"}], tag_field: None }`

Field	Explanation
<code>`name`</code>	The union's name (e.g., <code>"Data"</code>).
<code>`variants`</code>	Each field inside the union — these become the variants of the Rust <code>enum`</code> .
<code>`tag_field`</code>	<code>`Option`</code> — If the union has an associated "tag" field (like <code>`int tag;`</code>) to identify which variant is active, store its name here. Set to <code>`None`</code> during parsing; resolved later in Module 3.

StructAST — C struct definitions (Lines 41-45)

```
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct StructAST {
    pub name: String,
    pub fields: Vec<(String, String)>,
}
```

For struct `Point { int x; int y; };` StructAST { name: "Point", fields: [("x", "int"), ("y", "int")] }`

- `fields: Vec<(String, String)>` — A list of **tuples**. Each tuple is `(field_name, field_type)`. A tuple `(A, B)` groups two values together without naming them — simpler than a struct for two fields.

`TranslationUnitAST` — The Whole Program (Lines 47-63)

```
[derive(Debug, Clone, PartialEq, Eq)]
pub struct TranslationUnitAST {
    pub functions: Vec<FunctionAST>,
    pub unions: Vec<UnionAST>,
    pub structs: Vec<StructAST>,
}

impl TranslationUnitAST {
    pub fn new() -> Self {
        Self {
            functions: Vec::new(),
            unions: Vec::new(),
            structs: Vec::new(),
        }
    }
}
```

This is the ROOT of the AST — it represents the ENTIRE C source file. All functions, unions, and structs found in the file are collected here.

Field	What it holds
<code>`functions`</code>	Every function definition found in the C file.
<code>`unions`</code>	Every <code>`union { ... }`</code> definition found.
<code>`structs`</code>	Every <code>`struct { ... }`</code> definition found.

The `ASTError` Type (Lines 65-78)

```
[derive(Debug, Clone)]
pub enum ASTError {
    ParseFailed(String),
}

impl fmt::Display for ASTError { ... }
impl std::error::Error for ASTError {}
```

Only one error variant: parsing completely failed with a message. Follows the same pattern as `ValidationError` and `PreprocessorError`.

The Main Entry Point `construct_ast()` (Lines 80-95)

```

pub fn construct_ast(
    expanded: &ExpandedCSource
) -> Result<TranslationUnitAST, ASTError> {
    let content = &expanded.expanded_content;
    let mut ast = TranslationUnitAST::new();

    ast.unions      = parse_c_unions(content);
    ast.structs     = parse_c_structs(content);
    ast.functions   = parse_c_functions(content);

    Ok(ast)
}

```

Line	Code	Explanation
<code>`let content = &expanded.expanded_content`</code>	Get the text	Borrow just the expanded source string (don't clone the whole struct).
<code>`let mut ast = TranslationUnitAST::new()`</code>	Empty tree	Start with an empty AST that we'll fill in.
<code>`ast.unions = parse_c_unions(content)`</code>	Parse unions	Scan the text for <code>`union NAME {`</code> blocks and parse them.
<code>`ast.structs = parse_c_structs(content)`</code>	Parse structs	Same for <code>`struct NAME {`</code> blocks.
<code>`ast.functions = parse_c_functions(content)`</code>	Parse functions	Find and parse all function definitions.
<code>`Ok(ast)`</code>	Return success	The populated AST is returned.

`parse_c_unions()` — *How Union Parsing Works (Lines 97-142)*

```
fn parse_c_unions(content: &str) -> Vec<UnionAST> {
    let mut unions = Vec::new();
    let mut in_union = false;
    let mut current_name = String::new();
    let mut current_variants = Vec::new();

    for line in content.lines() {
        let trimmed = line.trim();
        if trimmed.starts_with("union ") && trimmed.contains('{') {
            in_union = true;
            let parts: Vec<&str> = trimmed.split_whitespace().collect();
            if parts.len() >= 2 {
                current_name = parts[1].trim_matches('{').to_string();
            }
            current_variants.clear();
            continue;
        }
        // ...
    }
}
```

State	Meaning
`in_union = false`	Currently reading top-level code (not inside a union).
`in_union = true`	We found `union Name {` and are now reading fields.
When we see `}`	The union is closed. Push a `UnionAST` into the result list. Reset state.

Field parsing inside union:

```
if trimmed.contains(';') {
    let parts: Vec<&str> = trimmed.trim_matches(';')
        .split_whitespace().collect();

    if parts.len() >= 2 {
        let field_type = parts[0].to_string();
        let field_name = parts[1].trim_matches('*').to_string();
        current_variants.push(UnionVariantAST { field_name, field_type });
    }
}
```

Code	Example: parses `int i;`
`trimmed.trim_matches(';')`	Strips the trailing semicolon → `int i`
`.split_whitespace().collect()`	Splits into `[ "int", "i" ]`
`parts[0]`	`"int"` — the type
`parts[1].trim_matches('*')`	`"i"` — the name (strips `*` if it's a pointer field)

`parse_c_structs()` follows IDENTICAL logic, just looking for `struct` instead of `union`.

parse_c_functions() — How Function Parsing Works (Lines 186-270)

```
fn parse_c_functions(content: &str) -> Vec<FunctionAST> {
    let mut functions = Vec::new();
    let lines: Vec<&str> = content.lines().collect();
    let mut i = 0;

    while i < lines.len() {
        let line = lines[i].trim();

        if line.contains('(')
            && line.contains(')')
            && (line.ends_with('{') || (i + 1 < lines.len()
                && lines[i + 1].trim() == "{"))
        {
            if let Some(func) = parse_single_function_signature(line, &lines, &mut i) {
                functions.push(func);
            }
        }
        i += 1;
    }
    functions
}
```

Function signature detection rule:

A line is a function signature if it:

Contains (and) — has a parameter list

Ends with { OR the NEXT line is a bare { — the function body starts

Code	Explanation
<code>`line.ends_with('{')`</code>	<code>`void swap(int **a, int **b) {`</code> — brace on same line
<code>`lines[i + 1].trim() == "{`</code>	<code>`void swap(...)`</code> followed by <code>`{`</code> on next line — both styles supported
<code>`parse_single_function_signature(line, &lines, &mut i)`</code>	Delegates to a helper that extracts the full function. Passes <code>`&mut i`</code> so the helper can advance the line counter past the function body.

parse_single_function_signature() — Extracting One Function (Lines 212-270)

```
fn parse_single_function_signature(
    sig_line: &str,
    lines: &[&str],
    line_idx: &mut usize,
) -> Option<FunctionAST> {
    let open_paren = sig_line.find('(')?;
    let close_paren = sig_line.rfind('')?;

    let head = sig_line[..open_paren].trim();
    let head_parts: Vec<&str> = head.split_whitespace().collect();
    if head_parts.len() < 2 { return None; }

    let return_type = head_parts[..head_parts.len() - 1].join(" ");
    let func_name = head_parts[head_parts.len() - 1]
        .trim_matches('*').to_string();

    let params_str = &sig_line[open_paren + 1..close_paren].trim();
    let parameters = parse_parameters(params_str);
    // ...
}
```

Step	Code	Example with <code>void swap(int **a, int **b) {</code>
Find <code>`(`</code>	<code>`sig_line.find('(')`</code>	Index = 9
Find last <code>`)`</code>	<code>`sig_line.rfind('')`</code>	Index = 25 (rfind finds the LAST, important for nested parens)
Everything before <code>`(`</code>	<code>`sig_line[..9]` = <code>"void swap"</code></code>	Split = <code>["void", "swap"]</code>
Return type	<code>`head_parts[..1].join(" ")` = <code>"void"</code></code>	Everything before the last word
Function name	<code>`head_parts.last()` = <code>"swap"</code></code>	Last word, trimming <code>`*`</code> if pointer return
Params string	<code>`sig_line[10..25]` = <code>"int **a, int **b"</code></code>	Between <code>`(`</code> and <code>`)`</code>

Body collection with brace counting:

```
let mut brace_count = 0i32;
let mut started = false;

for (curr_line_num, &raw_l) in lines.iter().enumerate().skip(start_line) {
    for c in l.chars() {
        if c == '{' { brace_count += 1; started = true; }
        else if c == '}' { brace_count -= 1; }
    }
    if started { body_lines.push(l.to_string()); }
    if brace_count == 0 && started {
        *line_idx = curr_line_num; // Tell the outer loop where we stopped
        break;
    }
}
```

Code	Explanation
<code>`brace_count`</code>	Goes up on `{`, down on `}`. When back to 0, the function body is complete.
<code>`started`</code>	Set to `true` when the first `{` is seen. Prevents counting braces before the function opens.
<code>`*line_idx = curr_line_num`</code>	Updates the OUTER loop's position through the <code>`&mut; usize`</code> . This skips past the entire function body in the outer scan.

parse_parameters() — Parsing Parameter Lists (Lines 272-307)

```
fn parse_parameters(params_str: &str) -> Vec<ParameterAST> {
    for param_decl in params_str.split(',') {
        let pointer_depth = trimmed.chars().filter(|&c| c == '*').count();
        let is_union = trimmed.contains("union ");
        let clean_decl = trimmed.replace('*', " ");
        let parts: Vec<&str> = clean_decl.split_whitespace().collect();
        // Last word = name, everything before = type
        let name = parts.last().unwrap().to_string();
        let base_type = parts[..parts.len() - 1].join(" ");
        params.push(ParameterAST { name, c_type: base_type, pointer_depth, is_union });
    }
}
```

	Code	Example with <code>`int **a`</code>		
	<code>`filter(\`</code>	<code>&c;\`</code>	<code>c == '*').count()\`</code>	<code>`2` stars four</code>
	<code>`replace('*', " ")`</code>	<code>`"int **a" → "int a"`</code>		
	<code>`split_whitespace()\`</code>	<code>`["int", "a"]`</code>		
	<code>`last()\`</code>	<code>`"a"`</code>		
	<code>`parts[..len-1].join(" ")`</code>	<code>`"int"`</code>		

parse_call_site() — Detecting Function Calls (Lines 309-329)

```
fn parse_call_site(line: &str, line_num: usize) -> Option<CallSiteAST> {
    let open_p = line.find('(')?;
    let close_p = line.rfind('')?;

    let left = line[..open_p].trim();
    let callee_parts: Vec<&str> = left.split_whitespace().collect();
    let callee = callee_parts.last()?
        .trim_matches(|c| c == '=' || c == '+' || c == '-')
        .to_string();

    if callee.is_empty() || callee == "printf" || callee == "scanf" {
        return None;
    }

    let args_str = &line[open_p + 1..close_p];
    let arguments: Vec<String> = args_str.split(',')
        .map(|s| s.trim().to_string())
        .collect();

    Some(CallSiteAST { callee, arguments, line: line_num })
}
```

— early exit			
= foo(x)`,			
re `(`.			
	c == '='	c == '+'	c == '-'`
	callee == "scanf"	Skip known C library functions — we don't want them in the call graph.	
].			
	s.trim().to_string())`	Trim whitespace from each argument.	

The Unit Test (Lines 331-367)

```
#[test]
fn test_function_and_call_site_parsing() {
    let code = r#"
void swap(int **a, int **b) {
    int *temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int x = 10;
    int y = 20;
    int *px = &x;
    int *py = &y;
    swap(&px, &py);
    return 0;
}
"#;

    let functions = parse_c_functions(code);
    assert_eq!(functions.len(), 2);

    let swap_fn = &functions[0];
    assert_eq!(swap_fn.name, "swap");
    assert_eq!(swap_fn.parameters.len(), 2);
    assert_eq!(swap_fn.parameters[0].pointer_depth, 2); // int **a

    let main_fn = &functions[1];
    assert_eq!(main_fn.name, "main");
    assert_eq!(main_fn.call_sites.len(), 1);
    assert_eq!(main_fn.call_sites[0].callee, "swap");
    assert_eq!(main_fn.call_sites[0].arguments, vec!["&px", "&py"]);
}
```

Assertion	What it verifies
<code>`functions.len() == 2`</code>	Two functions found: <code>`swap`</code> and <code>`main`</code> .
<code>`swap_fn.parameters[0].pointer_depth == 2`</code>	<code>`int **a`</code> correctly detected as depth 2.
<code>`main_fn.call_sites.len() == 1`</code>	Exactly one call site found inside <code>`main`</code> .
<code>`call_sites[0].callee == "swap"`</code>	The called function is <code>`swap`</code> .
<code>`call_sites[0].arguments == ["&px;", "&py;"]`</code>	Arguments captured correctly.

`r#"..."#` is a RAW STRING LITERAL in Rust. The `r#` prefix means special characters like `\n`, `"`, and `\` inside the string are treated literally, not as escape sequences. Perfect for embedding multi-line code snippets in tests!

How the Three Files Work Together

```

[mod.rs] preprocess_and_parse(validated: &ValidatedCSource)
|
| Step 1:
v
[preprocessor.rs] preprocess_c_code(validated)
|
| Try 1: GCC subprocess (gcc -E -P file.c)
| Try 2: Internal macro expander (if GCC fails)
|
v Returns: ExpandedCSource { expanded_content: String }
|
| Step 2:
v
[ast_parser.rs] construct_ast(&expanded)
|
| parse_c_unions()    -> Vec<UnionAST>
| parse_c_structs()   -> Vec<StructAST>
| parse_c_functions()-> Vec<FunctionAST>
|     (includes parse_parameters + parse_call_site)
|
v Returns: TranslationUnitAST { functions, unions, structs }
|
v
[mod.rs] Returns: ProcessedModule2Output { expanded_source, ast }

```

Key Rust Concepts Introduced in This Module

Concept	Example	Meaning
<code>`crate::`</code>	<code>`crate::module1_validation::ValidatedCSource`</code>	Absolute path from root of the project.
<code>`pub use`</code>	<code>`pub use ast_parser::FunctionAST`</code>	Re-export a type at a higher level.
<code>`std::process::Command`</code>	<code>`Command::new("gcc").arg("-E").output()`</code>	Run an external program as a child process.
<code>`String::from_utf8_lossy(&bytes;)`</code>	Read process output	Convert raw bytes to String, replacing invalid chars.
<code>`output.status.success()`</code>	Check exit code	Returns <code>`true`</code> if process exited with code 0.
<code>`str.find(c)`</code>	<code>`sig_line.find('(')`</code>	Find the index of the first occurrence of a char. Returns <code>`Option`</code> .
<code>`str.rfind(c)`</code>	<code>`sig_line.rfind(')`</code>	Find the LAST occurrence (right-find).
<code>`?` on <code>`Option`</code></code>	<code>`sig_line.find('(')?`</code>	Inside a function returning <code>`Option`</code> , <code>`?`</code> returns <code>`None`</code> immediately if the value is <code>`None`</code> .
<code>`Vec<&str;>` vs <code>`Vec`</code></code>	<code>`parts: Vec<&str;>`</code>	<code>`&str;`</code> is a borrowed string slice (no allocation); <code>`String`</code> is owned (allocated).

Concept	Example	Meaning
<code>`enumerate()`</code>	<code>`lines.iter().enumerate()`</code>	Produces <code>`(index, value)`</code> pairs.
<code>`skip(n)`</code>	<code>`enumerate().skip(start_line)`</code>	Skip the first <code>`n`</code> elements of an iterator.
Raw strings	<code>`r#"void swap..."#`</code>	String literal where <code>`\`</code> is not an escape.
<code>`&mut; usize`</code>	<code>`line_idx: &mut; usize`</code>	Mutable reference — the function can change the caller's variable.
<code>`*line_idx = ...`</code>	Dereference to write	Dereference the <code>`&mut; usize`</code> to write the new value into it.

Summary

Module 2 transforms raw validated C text into a structured, machine-readable AST in two stages:

Stage 2A — Preprocessing (`preprocessor.rs`):

- Tries to run system GCC (`gcc -E -P`) for best results
- Falls back to an internal line-by-line processor that handles `#define`, `#ifdef`, `#ifndef`, `#else`, `#endif`, and simple macro substitution
- Outputs `ExpandedCSource` — the expanded C code as a String

Stage 2B — AST Construction (`ast_parser.rs`):

- Parses the expanded C code line-by-line
- Builds `UnionAST` nodes for every `union { }` definition
- Builds `StructAST` nodes for every `struct { }` definition
- Builds `FunctionAST` nodes for every function, including:
 - `ParameterAST` list (name, type, pointer depth, union flag)
 - `CallSiteAST` list (which functions are called, with what arguments, on which line)

The resulting `TranslationUnitAST` is the foundation that ALL subsequent modules (3, 4, 5) use for their analysis and code generation.

Document generated for C2SafeRust-LLM Mini Project S3

Module 2: Preprocessing & AST Construction — Complete Line-by-Line Explanation